

From Gigabytes to Guidance: Efficiently Adapting a Compact LLM for STEM Education

Alix Papadatos | 347263 | alix.papadatos@epfl.ch
Inés Altemir Marinas | 344399 | ines.altemirmarinas@epfl.ch
Nizar Ben Mohamed | 330469 | nizar.benmohamed@epfl.ch
Florian Tanguy | 347510 | florian.tanguy@epfl.ch
Deep-learners

Abstract

In this project, we aim to transform the Qwen/Qwen3-0.6B-Base model into an effective AI tutor for advanced STEM subjects through a structured fine-tuning pipeline.

We begin with general supervised fine-tuning (SFT) using open-ended STEM questions to establish foundational domain knowledge and reasoning skills. Building on this base, we conduct task-specific instruction fine-tuning.

Our findings highlight the potential of using and training lightweight models, showing that they can be effectively adapted for educational settings with careful design and domain-specific alignment.

1 Introduction

In recent years, LLMs have rapidly evolved into powerful tools for a wide range of natural language understanding and generation tasks. However, aligning these models with domain-specific content—such as advanced STEM topics—remains a difficult and critical challenge. Current state-of-the-art LLMs often lack the depth of reasoning and factual precision required to perform well on high-level scientific tasks, particularly when dealing with fine-grained questions or complex multi-step reasoning.

Our project investigates how a compact open-source LLM, Qwen3-0.6B-Base, can be adapted into a capable and efficient AI tutor for STEM education. Unlike larger proprietary models such as GPT-4 or Claude, Qwen’s lightweight architecture is more accessible, but also demands more careful post-training to achieve competitive performance.

Our approach consists of a two-phase adaptation of Qwen. First, we fine-tuned the base model on a broad corpus of general STEM questions to imbue it with foundational scientific knowledge and reasoning ability. Then, we applied instruction fine-tuning depending on our model’s task. This

modular training strategy allowed us to develop specialized capabilities in each submodel while preserving a common knowledge base.

2 Approach

To develop a capable and instruction-aligned AI tutor for STEM education, our training pipeline was organized into two key stages: a general supervised fine-tuning (SFT) phase to instill foundational scientific reasoning followed by a task-specific instruction fine-tuning.

2.1 General Supervised Fine-Tuning

We began by fine-tuning the Qwen/Qwen3-0.6B-Base model on a diverse mixture of open-domain STEM datasets. These datasets included open-ended questions covering mathematics, physics, biology, computer science, and related fields. The goal of this stage was to provide the model with a broad foundation in scientific language and reasoning through open-form answers. The data included conceptual explanations and problem-solving processes.

This general SFT was conducted using a conservative training configuration: fewer epochs and a lower learning rate—to avoid overfitting and preserve the generalization capacity of the base model.

2.2 DPO Model

From this common SFT base, we pursued two parallel tracks. First, to create a preference-aligned conversational agent, we fine-tuned the SFT model using Direct Preference Optimization (DPO) (Rafailov et al., 2024). This process optimizes the policy model π_θ against a frozen reference π_{ref} by maximizing the margin between the log-probability ratios of chosen (y_w) over rejected (y_l) responses. This is formalized by minimizing the DPO loss $\hat{\mathcal{L}}_{\text{DPO}}$, defined using the reward term $\hat{r}_\theta(y, x)$, we first define the reward difference $\Delta \hat{r}_\theta$:

:

$$\Delta \hat{r}_\theta := \beta \log \left(\frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} \right) - \beta \log \left(\frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \quad (1)$$

$$\hat{\mathcal{L}}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) := -E_{(x, y_w, y_l) \sim \mathcal{D}} [\log \sigma(\Delta \hat{r}_\theta)] \quad (2)$$

2.3 MCQA Model

Using the general fine tuned model, we focused on building the MCQA model. This stage involved instruction fine-tuning on a dataset composed strictly of multiple-choice STEM questions. Each example was formatted such that the model’s target output consisted solely of the correct answer key (e.g., "A", "C", or a numerical value) along with the corresponding answer text. This structure aligns precisely with the final evaluation format, which tests for answer accuracy rather than reasoning or explanation.

To reinforce the model’s ability to learn this constrained output format, we trained using standard cross-entropy loss, with a higher learning rate and more training epochs than used during the initial SFT. This aggressive optimization strategy was intended to sharpen the model’s classification behavior and ensure confident, consistent predictions in response to MCQA inputs.

To further improve performance in underrepresented areas, we added a small synthetic dataset in a final fine-tuning stage. This data targeted specific STEM topics that were either not well covered in the training corpus or were identified as weak points through preliminary evaluation. This step helped improve model robustness and topic coverage before final evaluation.

2.4 Quantized Model

For computational efficiency, this fine-tuning was performed using Low-Rank Adaptation (LoRA) (?) with a rank of $r = 16$. In the second track, we fine-tuned the same SFT model for Multiple-Choice Question Answering (MCQA) by adding a classification head. To enhance the deployment feasibility of our team’s fine-tuned MCQA model, we applied Post-Training Quantization (PTQ). Our approach centers on the bitsandbytes library, as its focus on weight-only quantization proved most effective for our efficiency goals. We briefly explored alternative methods like optimum-quanto and llm-compressor, but found that their inclusion of activation quantization proved unsuitable,

leading to significantly higher VRAM usage and evaluation challenges in preliminary tests, resulting in a poor efficiency score. We therefore concentrated our efforts on optimizing weight-only configurations with bitsandbytes, evaluating both 8-bit and more aggressive 4-bit NormalFloat4 (NF4) techniques to find the best balance between performance and resource cost. All quantized models used a bfloat16 compute data type during matrix operations. Finally, the MCQA model was also integrated as the generator in a Retrieval-Augmented Generation (RAG) framework to produce more factually grounded responses.

2.5 RAG Model

Our RAG implementation adopts a retrieval-then-generate architecture that augments the pre-trained MCQA model with an external knowledge retrieval system, rather than training an end-to-end RAG model from scratch.

Therefore, the overall model consists of two distinct components: (1) a frozen retrieval encoder sentence-transformers/all-MiniLM-L6-v2) for document and query embedding, and (2) the pre-trained MCQA generator model. The retrieval process follows the standard dense passage retrieval paradigm where document similarity is computed via cosine similarity.

$$\text{sim}(q, d_i) = \frac{\mathbf{E}_{\text{query}}(q) \cdot \mathbf{E}_{\text{doc}}(d_i)}{\|\mathbf{E}_{\text{query}}(q)\|_2 \cdot \|\mathbf{E}_{\text{doc}}(d_i)\|_2}$$

With the limitation to employ a single embedding model for both queries and documents imposed, the methodology of freezing the document embedding and finetuning the query embedding model along the generator model, as done in (Lewis et al., 2021) was discarded.

If the encoder were to be finetuned during training, in a first instance, if done naively, it would necessitate re-embedding the entire 100k-chunk knowledge base after each training iteration, making the approach computationally prohibitive. Aware of these constraints, alternative strategies could have been implemented to mitigate computational costs, such as gradient accumulation with encoder updates every N steps, or partial re-embedding of documents at intervals. Nonetheless, we deliberately chose not to fine-tune the embedding model, as the sentence-transformers/all-MiniLM-L6-v2) model is already optimized for semantic similarity

tasks, making further fine-tuning likely to yield diminishing returns while introducing training complexity.

Regarding the augmentation strategy, rather than training the generator with retrieved contexts integrated into the training data, we employ inference-time augmentation. This approach leverages the model’s existing reasoning capabilities while providing access to external knowledge without requiring parameter updates, offering a simple and practical solution.

3 Experiments

- **Data:** For the initial SFT to get our base model, we used 12000 samples from 3 datasets: SirNeural/flan_v2 with 1000 samples, nvidia/OpenMathInstruct with 5000 samples and AI-MO/NuminaMath-1.5 with 6000 samples.

For the DPO model, we curated two STEM-focused datasets. For training, we filtered the argilla/ultrafeedback-multi-binarized-preferences-cleaned dataset (Bar-tolome et al., 2023) using a keyword-based approach to isolate relevant samples. For evaluation, we applied the same filtering method to the allenai/reward-bench dataset (Lambert et al., 2024).

For the MCQA model we use a mix from the following datasets: Idavidrein/gpqa with 448 samples, allenai/sciq with 500 samples, deepmind/aqua_rat with 100 samples, allenai/ai2_arc with 3370 samples and Synthetic dataset from chatGPT with 835 samples.

For the quantized mode, we evaluated all model variants on the **ARC-Easy** benchmark (Clark et al., 2018), using accuracy to measure task performance. To quantify efficiency, we recorded peak VRAM usage (GB), on-disk size (GB), and a custom score defined as **Score = Accuracy / VRAM** to assess the overall performance-to-cost trade-off.

For the RAG model, as external knowledge base (KB), we employ: *wiki_stem* and *camel-ai/physics*, with *wiki_stem* accounting for 77.8% of the total KB, and *camel-ai/physics* the rest. Our document processing pipeline preprocess a large external knowledge base by chunking documents into 256-token segments using the all-MiniLM-L6-v2 tokenizer. This

chunking strategy creates exactly separated chunks without overlap to maximize information density while respecting the model’s token limit. From the original document corpus, we extract the top 100k longest chunks to focus on the most information-rich content.

- **Evaluation method:** For the DPO model, our primary metric is Preference Accuracy. This measures the percentage of times the model correctly assigns a higher log-probability to the chosen response (y_w) over the rejected response (y_l) on our filtered evaluation set.

For the MCQA model, we used accuracy as our metric. We evaluated on the provided zechen-nlp/MNLP_STEM_mcqa_evals dataset, and the easy subset "ARC-Easy" of the allenai/ai2_arc dataset. The quantized and RAG model employs the same metrics as MCQA.

- **Baselines:** The baseline for our DPO model is its direct predecessor, the Nbenmo/M3_SFT model. This allows us to quantify the improvement gained specifically from the DPO alignment stage.

Similarly, for our MCQA model, the baseline are its 2 predecessors, the base Qwen model, and the Nbenmo/M3_SFT model.

For the quantized model and RAG model, the baseline is the original MCQA model.

- **Experimental details:** For the DPO model, we first conducted hyperparameter tuning on small data subsets, with results summarized in Table 1. The final model was trained for one epoch on the Gnoto platform with the optimal configuration: LoRA rank $r = 16$, DPO $\beta = 0.1$, and a learning rate of $2e-5$. We set `max_length=1024` and `max_prompt_length=384` based on our data distribution.

For our MCQA model, we compare our MCQA model with the performance of our 2 baseline models on the test set of the mentioned datasets. This will allow us to see the improvement from every fine tuning step.

¹ For the quantized model, the results of ap-

¹The evaluation for quantization used a specific configuration within the `lighteval` framework to accurately measure resource usage. This setup differ from the original MCQA model in this report, which explains the difference in accuracy.

Table 1: Hyperparameter Tuning for the DPO Model.

r	β	Learning Rate	Val. Accuracy
8	0.1	2e-5	79.45%
16	0.1	2e-5	82.20%
32	0.1	2e-5	80.10%
16	0.05	2e-5	81.15%
16	0.1	2e-5	82.20%
16	0.5	2e-5	78.30%
16	0.1	1e-5	80.55%
16	0.1	2e-5	82.20%
16	0.1	5e-5	79.70%

plying various quantization configurations to this model are summarized in Table 2.

Method	Acc. (%)	VRAM (GB)	Disk (GB)	Score
Original	67.21	8.242	2.22	8.16
8-bit (t=6.0)	67.38	3.357	0.70	20.07
8-bit (t=5.0)	67.34	3.357	0.70	20.06
8-bit (t=4.0)	67.17	3.357	0.70	20.01
4-bit NF4 (DQ)	66.04	3.107	0.50	21.25
4-bit NF4 (NoDQ)	66.12	3.126	0.52	21.15
4-bit FP4 (DQ)	61.32	3.338	0.50	18.37

Table 2: Quantization results. The score is Acc/VRAM. For 8-bit models, t=outlier threshold. DQ denotes Double Quantization.

• Results:

3.1 DPO model

The DPO model’s preference accuracy is detailed in Table 3. The training logs showed a consistent increase in reward accuracy, confirming effective learning.

Table 3: DPO Preference Accuracy vs. SFT Baseline.

Model	Preference Accuracy (%)
SFT Baseline	76.29
Our DPO Model	78.08

3.2 MCQA Model

The results are displays in Table 4

Model	Course provided data	ARC-Easy
Base Qwen	0.25	0.27
General SFT	0.35	0.59
Final MCQA model	0.38	0.72

Table 4: MCQA Model results. The score is accuracy

3.3 Quantization

The results show dramatic efficiency gains for all quantized models. Both 8-bit and 4-bit NF4 configurations successfully compressed the model with minimal impact on its reasoning accuracy.

3.4 RAG

4 Analysis

4.1 DPO Model Analysis

Qualitative analysis shows the DPO model succeeds on tasks with objective correctness criteria. It consistently identified subtle bugs in code, such as incorrect logic in C++ or variable-name typos in Python. This indicates DPO reinforced adherence to formal problem constraints.

The primary failure mode is a bias towards verbosity over conciseness. The model incorrectly preferred a descriptive narrative over a direct calculation in a math problem and a long-form explanation over the simple answer for a decimal conversion. A secondary failure occurred on a safety query, where the model preferred a speculative, hallucinated response over a correct refusal to answer, revealing a weakness in its safety alignment.

4.2 MCQA Model Analysis

Table 4 summarizes the performance of our MCQA model across two evaluation benchmarks.

We observe a clear upward trend in accuracy across all stages of training. The base Qwen model performs poorly on both datasets, achieving only 25% on the course dataset and 27% on ARC-Easy, which is close to random guessing for multiple-choice formats.

After the general supervised fine-tuning (General SFT), accuracy improves substantially—rising to 35% on the course dataset and 59% on ARC-Easy. This demonstrates the effectiveness of domain-specific instruction in enhancing general scientific reasoning capabilities.

The final instruction tuning stage, using MCQA-specific formatting and a higher learning rate, leads

to further improvements: 38% on the course dataset and a notable 72% on ARC-Easy. This suggests that structured answer formatting and targeted fine-tuning help the model better align with the answer-selection task.

Overall, the results validate our approach of multi-phase fine-tuning and highlight the benefits of task-aligned formatting and curriculum-style training progression.

4.3 Quantization

Our analysis of the results (Table 2) reveals that the fine-tuned model is highly resilient to quantization. The 8-bit model reduced VRAM usage by nearly 60% with no degradation in accuracy, confirming that this level of precision is sufficient to preserve the model’s learned parameters. While more aggressive, 4-bit quantization using the NF4 data type proved extremely effective, incurring only a minor 1 percentage point accuracy drop. In contrast, the standard FP4 data type was suboptimal, causing a significant 6-point performance loss. Ultimately, the **‘4-bit BNB (NF4, DQ)’** configuration provides the best overall trade-off, achieving the highest efficiency score (21.25). By reducing disk size by 77% and VRAM by 62% for a negligible performance cost, it stands as the ideal candidate for creating a lightweight, deployable AI tutor.

4.4 RAG Model

The RAG model demonstrates consistent performance improvements over the baseline MCQA model, achieving accuracy gains of 2-5% across multiple test runs on both the provided evaluation dataset and the ARC-Easy subset. These results align with our expectations and validate the effectiveness of our retrieval-augmented approach.

The moderate but consistent improvement suggests that external knowledge retrieval successfully addresses knowledge gaps present in the base model’s parametric memory. The 2-5% range indicates that while many questions can be answered using the model’s inherent knowledge, a meaningful subset benefits from external context. Furthermore, this modest improvement could be further analyzed for its diminishing returns, as augmenting an already capable MCQA model which possesses sufficient factual knowledge for most multiple-choice questions, might lead to a situation where retrieval may occasionally introduce noise rather than enhance reasoning.

The moderate gains also reflect potential domain misalignment between our external knowledge base and the specific question types in the evaluation datasets. While both sources cover STEM topics, the knowledge base may emphasize different subtopics, concepts, or levels of detail than those required for the multiple-choice questions, limiting the retrieval system’s ability to provide directly relevant context for every query.

5 Ethical considerations

As our project aims to develop an AI-based educational assistant, we must address its societal and ethical implications, focusing on accessibility, fairness, and potential misuse.

Multilingual Adaptation and Accessibility. While adapting our tutor to high-resource languages like French or German is feasible with new fine-tuning data, extending it to low-resource languages like Urdu or Swahili poses a major challenge. This would require substantial investment in data creation and collaboration with native experts to overcome data scarcity and the base model’s inherent limitations, preventing the deployment of a biased or ineffective tool.

Beneficiaries and Potential Harms. The intended beneficiaries are students, who gain a personalized learning aid. However, significant risks include fostering over-reliance that hinders critical thinking, generating plausible but factually incorrect information despite the RAG pipeline, and enabling academic dishonesty. Furthermore, the model may inherit and amplify subtle societal biases present in the large-scale datasets used for both pre-training and preference distillation.

Mitigation of Harms. To mitigate these harms, the tutor should be designed to guide students with questions rather than providing direct answers. Our RAG implementation is a primary mitigation against factual errors, and clear source citation is essential. Institutional policies must be established to govern the tool’s use and prevent academic misconduct. Crucially, the quantization work in this project directly promotes equity by lowering computational barriers, making the tutor accessible to students without high-end hardware.

Impact on Vulnerable Groups. The model’s EPFL-centric and English-dominant training data risks marginalizing students from different cultural and linguistic backgrounds. To address this, future iterations must consciously diversify the train-

ing and RAG documents. Implementing a robust user feedback mechanism is also critical to catch and correct biased, incorrect, or culturally insensitive responses, ensuring the tool serves all students fairly.

6 Conclusion

In this project, we successfully transformed the compact Qwen3-0.6B-Base model into a capable AI tutor for advanced STEM subjects using a structured fine-tuning pipeline. Our multi-stage approach, combining general supervised fine-tuning with task-specific instruction tuning for Multiple-Choice Question Answering (MCQA) and Direct Preference Optimization (DPO), yielded significant performance gains. The final MCQA model achieved 72% accuracy on the ARC-Easy benchmark, a dramatic improvement over the base model's 27%.

Our RAG implementation successfully demonstrates that augmenting a pre-trained MCQA model with external knowledge retrieval yields consistent accuracy improvements of 2-5%, validating the approach's effectiveness without requiring costly model retraining. While the modest gains reflect the inherent capabilities of modern language models and potential domain misalignment, the results establish RAG as a practical solution for enhancing knowledge-intensive question answering where factual recall is the primary limitation.

A key contribution of our work is demonstrating the efficacy of post-training quantization. We found that a 4-bit NF4 configuration reduced VRAM usage by 62% and on-disk size by 77% with a negligible drop in accuracy. This finding validates that specialized, lightweight models can be made highly efficient and accessible for real-world deployment on consumer hardware. While our Retrieval-Augmented Generation (RAG) framework lays the groundwork for improved factual reliability, future work should focus on refining model biases and expanding data diversity. Ultimately, our project confirms that with careful design, even small models can be adapted into powerful, efficient, and accessible educational tools.

References

Alvaro Bartolome, Gabriel Martin, and Daniel Vila. 2023. Notus. <https://github.com/argilla-io/notus>.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. [Think you have solved question answering? try arc, the ai2 reasoning challenge](#).

Nathan Lambert, Valentina Pyatkin, Jacob Morrison, LJ Miranda, Bill Yuchen Lin, Khyathi Chandu, Nouha Dziri, Sachin Kumar, Tom Zick, Yejin Choi, Noah A. Smith, and Hannaneh Hajishirzi. 2024. Rewardbench: Evaluating reward models for language modeling. <https://huggingface.co/spaces/allenai/reward-bench>.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2021. [Retrieval-augmented generation for knowledge-intensive nlp tasks](#).

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2024. [Direct preference optimization: Your language model is secretly a reward model](#).

A AI Usage Appendix

We used Google's large language model for assistance with coding and report writing. The team retained full intellectual ownership, and all AI-generated outputs were critically reviewed and manually verified by us.

- **Code Development:** The AI was used to refactor code for clarity, generate boilerplate, and suggest bug fixes. All suggestions were manually reviewed for correctness and then tested to ensure proper functionality.
- **Report Writing:** We authored all original content and analysis. The AI was then used to correct grammar and improve phrasing. We reviewed every edit to ensure our technical meaning and findings were preserved without alteration.

In all cases, the AI was used as a productivity tool, and the final responsibility for all code and text rests with the team.